SMART CHAT SYSTEM

TCP/UDP Socket Programming Project

Technical Project Report

Submitted By: Alok kumar sahu
Regd. number : 2402041118
Branch : CSE
Subject: Computer Networks

Table of Contents

1. Project Overview3
2. Objectives4
3. System Architecture4
4. Module Descriptions5
5. Protocol Design6
6. File Structure7
7. Technology Stack8
8. Frontend UI8
9. Setup & Usage Guide9
10. Port Configuration10
11. Testing & Scenarios10
12. Limitations & Future Work11
13. Conclusion11

1. Project Overview

The Smart Chat System is a multi-client real-time chat application built entirely using raw socket programming in Python. It demonstrates the practical use of both TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) within a single cohesive system, showcasing how the two transport-layer protocols complement each other in a networked application.

TCP is employed for reliable, ordered, and error-checked delivery of chat messages, ensuring every message reaches its destination intact and in sequence. UDP is employed for typing-status notifications, where low latency and high throughput matter more than guaranteed delivery — a missed "Alice is typing…" indicator is negligible, while a delayed one would degrade user experience.

The project has been extended with a browser-based frontend: a cyberpunk-themed HTML/CSS/JS interface communicates through a WebSocket bridge server that proxies messages bidirectionally between the browser and the existing TCP/UDP backend, requiring zero changes to the original Python code.

2. Objectives

- Implement multi-client chat using TCP socket programming in Python.
- Demonstrate real-time typing-status broadcast using UDP datagrams.
- Apply threading to handle concurrent client connections without blocking.
- Design a clean protocol using shared configuration constants.
- Bridge the terminal-based backend to a browser UI via WebSockets.
- Build an extreme-level animated frontend with a cyberpunk HUD aesthetic.
- Document the architecture and usage in a professional technical report.

3. System Architecture

The system follows a centralised server / multiple clients model. All communication is routed through the server; clients do not communicate directly with each other.

3.1 Architecture Overview

Layer	Component	Protocol
Backend Server	tcp_server.py + udp_server.py	TCP :9000 · UDP :9001
Bridge Layer	ws_bridge.py	WebSocket :8765
Frontend	index.html	HTTP :3000 / Browser
Terminal Clients	tcp_client.py + udp_client.py	TCP :9000 · UDP :9001

3.2 Data Flow

Chat Message Flow (TCP): Client Input → tcp_client.py → TCP socket → tcp_server.py → Broadcast → All other tcp_client.py instances.

Typing Status Flow (UDP): Keypress → udp_client.py → UDP datagram → udp_server.py → Broadcast → All other udp_client.py instances.

Browser Flow (WebSocket): Browser → WebSocket → ws_bridge.py → TCP/UDP → tcp_server.py / udp_server.py → ws_bridge.py → WebSocket → Browser.

4. Module Descriptions

4.1 common/config.py

Centralised configuration module shared by all other files. Defines the server host address, port numbers for TCP and UDP, buffer sizes, and protocol command tokens (/quit, TYPING, STOPPED). This single-source-of-truth design means changing a port requires editing only one file.

4.2 server/tcp_server.py

The TCP chat server. Creates a listening socket bound to HOST:TCP_PORT, sets SO_REUSEADDR to avoid port-reuse delays on restart, and enters an accept() loop. Each accepted connection spawns a dedicated daemon thread running handle_client(). A threading.Lock protects the shared clients dictionary from race conditions during concurrent reads and writes. The broadcast() function iterates over registered clients, skipping the sender, and calls sendall() to guarantee full message delivery.

4.3 server/udp_server.py

The UDP typing-status server. Uses a single blocking recvfrom() loop — no threads required — because UDP is stateless and each datagram is self-contained. Maintains a set of known client addresses and fans out TYPING / STOPPED status messages to every address except the sender. Unknown payloads are logged but discarded.

4.4 client/tcp_client.py

The TCP chat client. Connects to the server, completes the username handshake, then separates into two concurrent execution paths: a daemon thread running receive_messages() continuously reads from the socket and prints incoming data, while the main thread blocks on input() to read keyboard input and send it. A threading.Event coordinates shutdown between the two paths.

4.5 client/udp_client.py

The UDP typing-status client, encapsulated in the UDPClient class. Sends TYPING:<username> only once per typing session (deduplication prevents datagram flooding) and STOPPED:<username> when the user sends a message. A background thread with a 1-second socket timeout listens for status updates from the server and prints them inline.

4.6 server/ws_bridge.py

The WebSocket-to-TCP/UDP bridge. Built with the websockets async library. Each browser connection instantiates a ChatBridge object that holds both a TCP socket and a UDP socket, plus two background threads mirroring incoming TCP/UDP data to the WebSocket. The browser sends JSON payloads ({type, text}), which the bridge translates into raw socket calls; responses are translated back into JSON before being forwarded to the browser.

4.7 frontend/index.html

The single-file browser frontend. A live canvas draws falling matrix-style data streams and a glowing particle grid. The UI includes a cyberpunk HUD header with a real-time clock, an animated login card, a sidebar showing connected operators, a message panel with per-user colour-coded bubbles, a typing-indicator bar, and a particle-burst animation on every message send. All communication goes through the WebSocket bridge.

5. Protocol Design

5.1 TCP Message Protocol

Direction	Message Format	Description
Server → Client	"Enter your username: "	Initial handshake prompt
Client → Server	<username>	Username registration
Server → Client	"Welcome, <name>! …"	Join confirmation
Client → Server	<message text>	Chat message
Server → All	"[<username>]: <text>"	Broadcast to all peers
Client → Server	"/quit"	Graceful disconnect

5.2 UDP Status Protocol

Direction	Datagram Format	Description
Client → Server	"TYPING:<username>"	User started typing
Client → Server	"STOPPED:<username>"	User sent/stopped typing
Server → All	"[STATUS] <username> is typing…"	Broadcast to all peers

6. File Structure

File Path	Purpose
SMART CHAT SYSTEM/	Project root directory
common/__init__.py	Makes common a Python package
common/config.py	Shared constants: host, ports, tokens
server/__init__.py	Makes server a Python package
server/tcp_server.py	TCP server — accepts clients, broadcasts messages
server/udp_server.py	UDP server — broadcasts typing status
server/ws_bridge.py	WebSocket bridge for browser frontend
client/__init__.py	Makes client a Python package
client/tcp_client.py	TCP client — sends and receives chat messages
client/udp_client.py	UDP client — sends and receives typing status
frontend/index.html	Browser UI (single HTML file)
run_servers.py	Convenience launcher for TCP + UDP servers
README.md	Usage documentation

7. Technology Stack

Technology	Version	Usage
Python	3.10+	Core language for all server and client modules
socket (stdlib)	Built-in	Raw TCP and UDP socket programming
threading (stdlib)	Built-in	Concurrent client handling and I/O loops
asyncio (stdlib)	Built-in	Async WebSocket bridge event loop
websockets	11+	WebSocket server in ws_bridge.py
HTML5 / CSS3	Modern	Browser frontend structure and styling
JavaScript (ES6+)	Modern	WebSocket client, canvas animation, UI logic
Canvas API	Modern	Real-time matrix/particle background animation
Google Fonts	CDN	Orbitron, Rajdhani, Share Tech Mono typefaces

8. Frontend UI

The browser frontend uses a cyberpunk HUD aesthetic with a near-black void background, neon cyan and magenta accents, and monospace typography from the Orbitron and Share Tech Mono font families.

8.1 Visual Features

- Live canvas background with falling matrix-style character streams (TCP/UDP/hex characters and katakana) and a glowing particle grid — all rendered in real-time using the HTML5 Canvas API.
- CRT scanline overlay using CSS repeating-linear-gradient animated vertically.
- Four-corner bracket decorations giving a military-terminal feel.
- Animated header scan line that sweeps the full width using a CSS pseudo-element.
- Glitch effect on the hero title that fires every 8 seconds — text splits into cyan/magenta offsets.
- Gradient shimmer continuously sweeping across the logo text.

8.2 Interaction Animations

- Particle burst: 14 cyan particles explode outward from the Send button on every message.
- Message bubbles spring in with a scale + translateY cubic-bezier animation.
- Full-screen connecting overlay with spinner and blinking "ESTABLISHING LINK…" text.
- Typing indicator with three bouncing dots and staggered animation delays.
- Button hover: cyan flood-fill sweeps from left using a translateX pseudo-element.
- Input focus: border glows cyan with box-shadow and a scan underline grows from left.

9. Setup & Usage Guide

9.1 Prerequisites

- Python 3.10 or higher installed.
- pip install websockets (one external dependency).
- A modern web browser (Chrome, Firefox, Edge).

9.2 Running the System

Step	Terminal	Command
1	Terminal 1	python run_servers.py
2	Terminal 2	python server/ws_bridge.py
3	Terminal 3	python -m http.server 3000 --directory frontend
4	Browser	Open http://localhost:3000
5	Browser	Enter username → Click ESTABLISH CONNECTION

9.3 Commands

Command	Effect
/quit	Gracefully disconnect from the server and notify all other clients
(any text)	Broadcast the message to all currently connected clients
DISCONNECT button	Browser: sends /quit and closes the WebSocket connection

10. Port Configuration

Service	Port	Protocol	Managed By
TCP Chat Server	9000	TCP	run_servers.py → tcp_server.py
UDP Status Server	9001	UDP	run_servers.py → udp_server.py
WebSocket Bridge	8765	WebSocket	ws_bridge.py
Frontend HTTP Server	3000	HTTP	python -m http.server

11. Testing & Scenarios

11.1 Multi-Client Chat Test

1. Start run_servers.py.
2. Open three terminal windows and run python client/tcp_client.py in each.
3. Assign usernames Alice, Bob, and Charlie.
4. Send messages from each client and verify all others receive them.
5. Type /quit in one terminal and confirm the other two see the leave notification.

11.2 Typing Indicator Test

1. With two clients connected, begin typing in Client A without pressing Enter.
2. Verify Client B sees "Alice is typing…" within 1–2 seconds.
3. Send the message and confirm Client B sees "Alice stopped typing." followed by the message.

11.3 Browser Frontend Test

1. Start all three servers (Terminal 1: run_servers.py, Terminal 2: ws_bridge.py, Terminal 3: http.server).
2. Open index.html in two browser tabs simultaneously.
3. Enter different usernames in each tab and verify bidirectional messaging works.
4. Verify the online sidebar updates as users join and leave.
5. Confirm the typing indicator appears in one tab when the other is typing.

12. Limitations & Future Work

12.1 Current Limitations

- No message persistence — all messages are lost when the server restarts.
- No authentication — any username can be chosen without verification.
- Single-channel — there is no support for multiple chat rooms.
- No end-to-end encryption — messages travel in plaintext over the local network.
- The WebSocket bridge is single-threaded per client; under heavy load it may bottleneck.

12.2 Future Enhancements

- SQLite or Redis message persistence for chat history.
- Username password authentication with bcrypt hashing.
- Multiple named channels / rooms.
- TLS/SSL encryption for both TCP connections and WebSocket (wss://).
- File and image sharing over TCP.
- User presence states (away, do not disturb).
- Mobile-responsive frontend with PWA support.

13. Conclusion

The Smart Chat System successfully demonstrates the complementary use of TCP and UDP transport protocols within a single real-world application. TCP provides the reliability guarantees required for chat messages, while UDP delivers the low-latency, best-effort semantics appropriate for ephemeral typing indicators.

The project exercises key networking concepts including socket creation and binding, multi-threaded server design, race-condition prevention with locks, graceful connection teardown, and bridging between protocol layers. The WebSocket bridge extends the system to browser clients without modifying any backend code, illustrating good separation of concerns.

The cyberpunk browser frontend demonstrates how a technically rich backend can be paired with a visually striking interface, making the project suitable for both academic submission and portfolio demonstration.

END OF REPORT — SMART CHAT SYSTEM